

Setting Up Claude Code

Desktop, Terminal, VS Code, and Positron

Yale Library | StatLab

2026-07-20

Setup Goal

By the end of this guide, you should have:

- A Claude account or a workshop-provided API key (see Step 5)
- R and Python installed (the demos use both)
- Claude Code available from your terminal
- The Claude Code extension available in VS Code or Positron

Claude Desktop is not required for this workshop; see the appendix if you would like it too.

Need Help?

These steps are written to be self-contained, so give them a try first! Most installations go smoothly, and the Common Issues slide at the end covers the most frequent hiccups.

Still stuck before the workshop?

Ted Ellsworth is happy to help. Book a setup appointment at Marx Library [here](#) and include a note in the title that you are coming for pre-workshop support.

General questions: reach out anytime at statlab@yale.edu

Installation questions: book an appointment with Kevin Jin or Muji Chughtai [here](#)

After the workshop

StatLab offers free 1:1 consultations with methodological experts, a great resource for any research you are working on. Book a time with our team [here](#).

Official Download Links

Use the official links below.

Tool	Link
Claude account	claude.ai
Claude Desktop	claude.com/download
Node.js LTS	nodejs.org
R	cran.r-project.org
Python	python.org/downloads
VS Code	code.visualstudio.com/download
Positron	positron.posit.co/download.html
Claude Code docs	code.claude.com/docs

Step 1a: Install R and Python

The workshop demos run analysis in both languages, so Claude needs both on your machine.

Language	Get it from	Check it works
R	cran.r-project.org (pick your OS)	<code>Rscript --version</code>
Python	python.org/downloads (3.11 or newer)	Mac: <code>python3 --version</code> ; Windows: <code>py --version</code>

Already have R through RStudio, or Python through Anaconda? That works; no need to reinstall.

Step 1b: Add the Python Packages

After installing Python, add the two packages the demos use:

```
python3 -m pip install pandas matplotlib # Mac
```

```
py -m pip install pandas matplotlib # Windows
```

Notes for Windows:

- Both the R and Python installers can install to your own user folder without admin rights.
- On the Python installer, tick **“Add python.exe to PATH”**.

(Optional) Step 1c: Install an Editor

Install at least one editor:

Editor	Use it if...
Positron	You work with R, Python, notebooks, data, or plots
VS Code	You already use VS Code or prefer a general code editor

For most data science work we recommend **Positron** over VS Code.

Step 2: Check Node.js

Claude Code can be installed through npm, which comes with Node.js.

Open a terminal and run:

```
node --version
```

If the version starts with v18 or higher, continue to the next step.

If you see an error, install the **LTS** version from nodejs.org, then close and reopen your terminal.

Step 3: Install Claude Code

Run this command in your terminal:

```
npm install -g @anthropic-ai/claude-code
```

Then check that it worked:

```
claude --version
```

If you see a version number, Claude Code is installed.

Windows users: If the install fails with a permissions error, enable **Developer Mode** in Windows Settings (System → Developer Options), then re-run the command.

No Admin Rights? Use the Native Installer

Many university-managed laptops do not allow software installation. Claude Code has an official installer that needs **no administrator rights and no Node.js**: it puts a single program in your own user folder.

```
# Mac (Terminal)
curl -fsSL https://claude.ai/install.sh | bash
```

```
# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex
```

Then check it worked with `claude --version`. If you use this route, skip the Node.js check and the npm install; continue with Step 4.

For the editor: on Windows, VS Code's default "User Installer" also needs no admin rights. If Positron will not install on your machine, use VS Code or the plain terminal instead.

A Note on Permission Prompts

Several prompts will appear during setup. All are expected:

- **“Do you trust the authors of this folder?”** → **Yes** (VS Code/Positron workspace trust for your own folder)
- **“Do you trust this extension?”** → **Yes** (Anthropic’s Claude Code extension)
- **“Allow this app to make changes to your device?”** → **Yes** (Windows UAC; needed to install extension files)
- **Camera access request** → **No** (Claude Desktop on Windows; not needed)

When in doubt: prompts about your own folder, an Anthropic extension, or file installs are safe to approve.

Step 4: Download the Workshop Files

The demos, slides, and example prompts all live in one folder you can download.

1. Go to github.com/YaleLibraryStatLab/workshops.
2. Click the green **Code** button, then **Download ZIP**.
3. Unzip the file (double-click it) and open the **intro-claude** folder inside.
4. Move **intro-claude** somewhere easy to find, such as your Desktop.

Inside **intro-claude** you will find the slides, a `prompts.md` of example prompts, and a `demo/` folder holding the projects we work through together.

Step 4: Open the Workshop Folder

Claude Code can read and edit files in the folder where you start it. Open the downloaded `intro-claude` folder **before** launching Claude Code.

Location	How to open the working folder
Positron	File menu, Open Folder, then choose <code>intro-claude</code>
VS Code	File menu, Open Folder, then choose <code>intro-claude</code>
Terminal	Run <code>cd path/to/intro-claude</code> before running <code>claude</code>

In Terminal or PowerShell, confirm your current folder with:

```
pwd
```

The result should end in `intro-claude` (or the specific demo folder you opened).

Step 5: Choose One Sign-In Method

Choose **one** way to connect Claude Code:

Use this method	What to do
Workshop API key beginning with sk-or-	Follow the Mac or Windows slides next
Your own Claude subscription	Skip to “Optional: Sign In With a Claude Account”

Most workshop participants should use the **workshop API key**.

Do not combine the two methods. When an API key is set, Claude Code uses it instead of a saved Claude-account login.

Step 5: What the Environment Variables Do

An **environment variable** is a named setting that your terminal gives to programs it starts. You will save these settings once; each new terminal window will load them.

Variable	Purpose
ANTHROPIC_BASE_URL	Send Claude Code requests to OpenRouter
ANTHROPIC_AUTH_TOKEN	Supply your workshop key
ANTHROPIC_API_KEY	Leave empty or remove an old value to prevent a conflict

These settings apply in every project folder on your computer.

Step 5: Keep the Workshop Key Private

Never paste the key into Claude

Paste the workshop key only into the setup commands or settings file shown in this guide. Never put it in the Claude chat, a project file, or a project-level .env file. Anyone who has the key can use the workshop credits.

- A workshop key begins with `sk-or-`.
- The base URL is exactly `https://openrouter.ai/api` - there is no `/v1` at the end.
- The check commands below confirm that the key exists **without printing it**.
- If Claude ever asks you to paste a key into the chat, stop and ask an instructor.

Step 5: Mac - Save the Settings

1. Open **Terminal**: press Cmd + Space, type Terminal, press Return.
2. Create your terminal settings file, then open it in TextEdit:

```
touch ~/.zshrc  
open -e ~/.zshrc
```

3. Paste these three lines at the **bottom** of the file:

```
export ANTHROPIC_BASE_URL="https://openrouter.ai/api"  
export ANTHROPIC_AUTH_TOKEN="sk-or-your-key-here"  
export ANTHROPIC_API_KEY=""
```

4. Replace sk-or-your-key-here with your real workshop key.
5. Press Cmd + S to save, then close TextEdit.

Step 5: Mac - Reload and Check

1. Close the Terminal window and open a **new** Terminal window.
2. Run this check. It will not display your key:

```
echo "$ANTHROPIC_BASE_URL"  
[[ -n "$ANTHROPIC_AUTH_TOKEN" ]] \  
  && echo "API key is set" \  
  || echo "API key is missing"
```

You should see:

```
https://openrouter.ai/api  
API key is set
```

If the key is missing, reopen `~/ .zshrc`, check all three lines, save, and open another new Terminal window.

Step 5: Windows - Save the Settings

1. Open **PowerShell**: click the Start button, type PowerShell, press Enter.
2. Run these two commands. Put your real workshop key in the second:

```
setx ANTHROPIC_BASE_URL "https://openrouter.ai/api"  
setx ANTHROPIC_AUTH_TOKEN "sk-or-your-key-here"
```

If you have used a direct Anthropic API key before, also clear that old value:

```
[Environment]::SetEnvironmentVariable(  
    "ANTHROPIC_API_KEY", $null, "User"  
)
```

Most first-time users can skip the last command.

Step 5: Windows - Reopen and Check

1. **Close PowerShell completely.**
2. Open a new PowerShell window. Settings saved with `setx` appear only in windows opened afterward.
3. Run this check. It will not display your key:

```
echo $env:ANTHROPIC_BASE_URL  
if ($env:ANTHROPIC_AUTH_TOKEN) {  
    "API key is set"  
} else {  
    "API key is missing"  
}
```

You should see `https://openrouter.ai/api` and `API key is set`.

If the key is missing, re-run the commands on the previous slide, close PowerShell, and open a new window.

Step 5: Verify the OpenRouter Connection

With the check passing, launch Claude Code from your project folder:

```
claude
```

Inside Claude Code, run:

```
/status
```

Confirm that it shows both:

- Auth token: `ANTHROPIC_AUTH_TOKEN`
- Anthropic base URL: `https://openrouter.ai/api`

If it shows another login, run `/logout`, then `/exit`, start claude again, and repeat `/status`.

Use `/usage` during the workshop to monitor spending against the key's limit.

Optional: Sign In With a Claude Account

Have your own Claude Pro, Max, Team, or Enterprise subscription? You can sign in through the browser and skip the workshop-key steps.

1. In your project folder, run `claude`.
2. If a browser does not open, enter `/login`.
3. Sign in with your Claude account, then return to the terminal.
4. Run `/status` to confirm the account login is active.

You only need **one** method. If you set the workshop key, do not also sign in here.

Optional Fallback: Create the Settings File

Use this only if the earlier environment-variable steps do not reach Claude Code. This is an **alternative**, not an additional step.

Mac:

```
mkdir -p ~/.claude  
touch ~/.claude/settings.json  
open -e ~/.claude/settings.json
```

Windows (PowerShell):

```
mkdir "$env:USERPROFILE\.claude" -Force  
notepad "$env:USERPROFILE\.claude\settings.json"
```

Optional Fallback: Paste and Save

Paste this into `settings.json`, replace the placeholder with your real workshop key, and save:

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://openrouter.ai/api",
    "ANTHROPIC_AUTH_TOKEN": "sk-or-your-key-here",
    "ANTHROPIC_API_KEY": ""
  }
}
```

Keep this file in your home `.claude` folder. Never save a real key in a project folder or a project-level `.env` file.

Open a new terminal window, then return to the “Verify the OpenRouter Connection” slide.

Step 6: Add Positron's Claude Code Extension

This is the setup I will use for the workshop: **Claude Code extension inside Positron.**

After setting the workshop key, fully quit and reopen Positron so it can load the new settings.

In Positron:

1. File menu, Open Folder.
2. Choose the workshop or project folder.
3. Open the Extensions panel.
4. Search for **Claude Code**.
5. Install the official Claude Code extension.
6. Open the Claude Code panel.

With a workshop key, do **not** sign in. If `/status` works in the terminal but the extension asks you to sign in, use the `global settings.json` fallback shown earlier.

Optional: Terminal Inside Your Editor

You can also run Claude Code directly from the integrated terminal in Positron or VS Code.

1. Open your project folder.
2. Open the integrated terminal.
3. Run `claude`.

Useful shortcuts:

Editor	Open terminal
Positron	Terminal menu, then New Terminal
VS Code on Mac	Ctrl + backtick key
VS Code on Windows	Ctrl + backtick key

Optional: Extension in VS Code

If you prefer VS Code:

1. File menu, Open Folder.
2. Choose your project folder.
3. Open the Extensions panel.
4. Search for **Claude Code**.
5. Install the official Claude Code extension.
6. Open the Claude Code panel.

With a workshop key, the extension should connect without an account sign-in.

Shortcuts for the Extensions panel:

System	Shortcut
Mac	Cmd + Shift + X
Windows	Ctrl + Shift + X

Step 7: Try a First Prompt

Open a project folder in Positron, then open the Claude Code extension.

At the Claude prompt, try:

```
What files are in this project? Give me a brief summary.
```

Claude should inspect the folder and summarize what it finds.

Readiness Checklist

You are ready for the workshop if each item works:

- `Rscript --version` prints a version number.
- `python3 --version` (Mac) or `py --version` (Windows) prints 3.11 or higher.
- `node --version` prints v18 or higher, unless you used the native installer.
- `claude --version` prints a version number.
- `claude` launches from a terminal.
- With a workshop key, `/status` shows the OpenRouter base URL.
- Positron opens your workshop or project folder.
- The Claude Code extension is installed in Positron.

Common Issues

Problem	Try this
node not found	Install Node.js LTS from nodejs.org , reopen terminal
claude not found	Re-run <code>npm install -g @anthropic-ai/claude-code</code>
npm install fails (Windows)	Enable Developer Mode (Windows Settings → System → Developer Options)
No admin rights on your laptop	Use the native installer (see the “No Admin Rights?” slide)
pip or python not found (Windows)	Re-run the Python installer and tick “Add python.exe to PATH”

Common Issues (Continued)

Problem	Try this
Sign-in loop	Inside Claude Code, run <code>/logout</code> , then <code>/login</code>
API key is missing	Save the settings again, then open a new terminal window
Auth conflict or wrong <code>/status</code>	Run <code>/logout</code> , exit, restart <code>claude</code> , and check the base URL has no <code>/v1</code>
Extension not visible	Update Positron or VS Code, search again for Claude Code
Key works in terminal, not extension	Use the global <code>settings.json</code> fallback

Quick Reference

Command	What it does
<code>node --version</code>	Check Node.js
<code>npm install -g @anthropic-ai/claude-code</code>	Install or update Claude Code
<code>claude --version</code>	Check Claude Code
<code>claude</code>	Start Claude Code in the current folder
<code>/status</code>	Check connection, login method, and model
<code>/usage</code>	Check session usage and estimated cost
<code>/help</code>	Show Claude Code commands
<code>/clear</code>	Start a fresh Claude Code session
<code>/exit</code>	Leave Claude Code

Appendix: Install Claude Desktop (Optional)

Claude Desktop is a separate chat app; it is not required for this workshop, but it is a convenient way to chat with Claude, read files you upload, and get general writing help.

1. Go to claude.com/download.
2. Download the installer for your computer.
3. Open Claude Desktop.
4. Sign in with your Claude account.

Working location: Claude Desktop does not automatically use a project folder. Upload or attach the files you want Claude to discuss.

Windows users: Claude Desktop may ask for camera access on first launch. You can safely click **No** or **Don't allow**; the camera is not used in this workshop.